

Esame OS Internals (Cabodi)

Data: 10/07/2020

QUESTIONS

SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Memoria Virtuale, Working Set e Codice C)

Contesto: Sia dato il frammento di programma rappresentato:

```
#define N 512
float M[2*N][N], V[2*N*N];
int i, j, k, t;

for (i=t=0; i<2*N; i++) {
    k = (i<N) ? N-i : i-N+1;
    for (j=0; j<k; j++) {
        V[t++] = M[i][j];
    }
}
```

Il codice macchina generato da tali istruzioni viene eseguito in un sistema con gestione della memoria basata su demand paging (paginazione a richiesta), pagine da 2 KByte, utilizzando come politica di sostituzione pagine la "working set" (versione esatta) con $\delta = 10$. Si sa che:

- la dimensione di un `float` è di 32 bit;
- il segmento di codice (istruzioni in codice macchina) ha dimensioni inferiori a una pagina;
- `M` e `V` sono allocati ad indirizzi logici contigui, a partire dall'indirizzo logico `0x74240A00`;
- la matrice `M` è allocata seguendo la strategia "row major".

Si risponda alle seguenti domande:

- Quante pagine (e frame) sono necessarie per contenere la matrice `M` e il vettore `V`?
- Ipotizzando che le variabili `i`, `j`, `k`, `t` siano allocate in registri della CPU, quanti accessi in memoria (in lettura e scrittura) fa il programma proposto per accedere a dati (non vanno conteggiati gli accessi a istruzioni)?
- Calcolare il numero di page fault generati dal programma proposto, supponendo che siano allocati per esso 10 frame, di cui uno utilizzato (già all'inizio dell'esecuzione) per le istruzioni. (Considerare caso minimo e massimo, e motivare le risposte).

Domanda 2 (RAID e Affidabilità)

Contesto: Sia dato un disco organizzato con struttura RAID. Sono dati il tempo medio tra guasti (MTTF), il tempo medio per riparazione (MTTR), e il tempo medio tra perdite di dato (MTTDL).

Si risponda alle seguenti domande:

-) E' possibile che $MTTR > MTTF$?
 -) E' possibile che $MTTDL > 10 \cdot MTTF$?
 -) Si consideri una struttura RAID con 2 dischi in configurazione "mirrored". Se ognuno dei dischi può guastarsi in modo indipendente dall'altro, con $MTTF = 20000$ ore, quanto deve essere $MTTR$ per garantire $MTTDL > 100 \cdot MTTF$?
-

Domanda 3 (I/O Sincrono, Asincrono e Polling)

Si risponda alle seguenti domande:

-) Si spieghino le principali differenze tra gli I/O dei due tipi (sincrono e asincrono). Si considerino poi I/O bloccante e non bloccante: si tratta di sinonimi di asincrono e sincrono? Ci sono differenze? (se no, perché; se sì, quali?).
-) Un I/O sincrono può essere effettuato in polling oppure è necessario effettuarlo in interrupt? (Si risponda alla stessa domanda per il caso dell'I/O asincrono).
-) In quale modo può un processo beneficiare di un I/O asincrono? E' possibile, nel caso di I/O asincrono, scrivere un programma con istruzioni, successive all'I/O, che dipendano dai dati coinvolti (ad es. letti) durante l'I/O?

SECTION 2: OS161

Domanda 4 (System Call `sys_read` e Sicurezza)

Contesto: Sia dato un sistema OS161. Si consideri la funzione `sys_read`, utilizzata per realizzare la system call `read`. Se ne propone una realizzazione, limitata al caso di standard input:

```
int sys_read(int fd, userptr_t buf, int size) {
    if (fd == STDIN_FILENO) {
        kgets ((char *)buf, size);
        return size;
    } else {
        kprintf("sys_read implemented only on stdio\n");
        return -1;
    }
}
```

Si risponda alle seguenti domande:

-) E' possibile che la funzione sia richiamata direttamente da un processo user? (Motivare).
 -) La funzione è errata, in quanto la chiamata a `kgets`, così come effettuata, rischia di fare un accesso scorretto in memoria. Si spieghi il perché e si proponga una possibile correzione al problema.
 -) Perché la funzione ritorna `size`, qualora non ci sia errore? E' possibile che, in una realizzazione più completa della funzione, il valore ritornato sia maggiore o minore di `size`?
-

Domanda 5 (Sincronizzazione Multi-core e Semantica dei Semafori)

Contesto: Sia dato un sistema operativo OS161.

Si risponda alle seguenti domande:

-) Si spieghi perché, in un contesto multi-core (sono presenti più CPU) non è possibile realizzare la mutua esclusione semplicemente disabilitando e riabilitando l'interrupt.
-) Dato il codice delle funzioni di semaforo P e V:

```
void P(struct semaphore *sem) {
    spinlock_acquire(&sem->sem_lock);
    while (sem->sem_count == 0) {
        wchan_sleep(sem->sem_wchan, &sem->sem_lock);
    }
    sem->sem_count--;
    spinlock_release(&sem->sem_lock);
}
```

```
void V(struct semaphore *sem) {
    spinlock_acquire(&sem->sem_lock);
    sem->sem_count++;
    KASSERT(sem->sem_count > 0);
}
```

```
wchan_wakeone(sem->sem_wchan, &sem->sem_lock);  
spinlock_release(&sem->sem_lock);  
}
```

Rispondere ai quesiti:

1. A cosa serve lo spinlock (in entrambe le funzioni)?
2. Perché la `P` contiene un ciclo `while` anziché un `if`?
3. Perché la `wchan_sleep` riceve come parametro lo spinlock?
4. E' possibile che la chiamata alla `wchan_wakeone` svegli più di un thread in attesa?
) Spiegare brevemente la differenza tra condition variable e `wait_channel`.

ANSWERS

Domanda 1 (Memoria Virtuale, Working Set e Codice C)

Dati Iniziali:

- $N = 512$.
- Matrice M di dimensioni $[2N][N] \implies 1024 \times 512$ elementi `float`.
- Vettore V di dimensione $[2 \cdot N \cdot N] \implies 524288$ elementi `float`.
- Dimensione `float` = 4 Byte.
- Dimensione Pagina = 2 KB (2048 Byte).
- Indirizzo logico di partenza: `0x74240A00`.

A) Pagine e frame necessari per contenere M e V :

Calcoliamo lo spazio totale occupato in memoria:

- Dimensione di $M = 1024 \cdot 512 \cdot 4 \text{ Byte} = 2.097.152 \text{ Byte} = 2 \text{ MB}$.
- Dimensione di V (totale dichiarato) = $524288 \cdot 4 \text{ Byte} = 2.097.152 \text{ Byte} = 2 \text{ MB}$.
- Spazio totale contiguo = $4 \text{ MB} = 4.194.304 \text{ Byte}$.

Verifichiamo l'allineamento dell'indirizzo iniziale rispetto alla pagina da 2 KB (`0x800`): L'indirizzo `0x74240A00` ha un offset interno alla pagina pari a:

$$0x0A00 \pmod{0x0800} = 0x0200 = 512 \text{ Byte}$$

Le strutture dati iniziano al byte 512 della prima pagina e si estendono per 4.194.304 byte. L'ultimo byte si troverà a un offset complessivo dall'inizio della prima pagina pari a $512 + 4.194.304 - 1 = 4.194.815$. Calcoliamo il numero di pagine attraversate:

$$\lfloor 4.194.815 / 2048 \rfloor + 1 = 2048 + 1 = \mathbf{2049} \text{ pagine}$$

Per contenere interamente M e V contemporaneamente, sono necessari **2049 frame fisici**.

B) Accessi a memoria (dati):

Il programma calcola k in modo deterministico:

- Per i da 0 a 511: $k = 512 - i$. La somma delle iterazioni interne è $512 + 511 + \dots + 1 = 131.328$.
- Per i da 512 a 1023: $k = i - 512 + 1$. La somma delle iterazioni interne è $1 + 2 + \dots + 512 = 131.328$.

Il loop più interno esegue in totale $131.328 + 131.328 = \mathbf{262.656}$ iterazioni.

In ogni iterazione viene eseguita l'istruzione `V[t++] = M[i][j]`, che comporta:

- 1 lettura in memoria per $M[i][j]$
- 1 scrittura in memoria per $V[t]$

Totale accessi = $262.656 \times 2 = \mathbf{525.312}$ accessi.

C) Numero di Page Fault (Caso minimo e massimo):

Il flusso del programma è strettamente deterministico. **Il caso minimo e il caso massimo coincidono** in quanto non vi sono diramazioni condizionali dipendenti dai dati in input.

Ogni riga di M ha 512 elementi (2048 Byte), occupando esattamente lo spazio di una pagina. Poiché M è traslata di 512 Byte, la riga i inizia sempre a offset 512 della Pagina i .

- **Page Fault per M:** Si accede sempre a $M[i][0]$ per ogni riga, quindi si tocca l'inizio di ogni riga. L'ultima riga ($i = 1023$) legge fino a $k = 512$ elementi, scavalcando il limite della Pagina 1023 e toccando la Pagina 1024. Totale pagine distinte toccate da $M = \mathbf{1025}$ pagine (dalla 0 alla 1024).
- **Page Fault per V:** V inizia a offset 512 della Pagina 1024. Vengono scritti sequenzialmente 262.656 elementi (1.050.624 Byte). Tali byte attraversano $\lceil 1.050.624/2048 \rceil = \mathbf{514}$ pagine (dalla 1024 alla 1537).
- **Pagine Condivise:** L'unica pagina in comune è la **1024**. Viene caricata da V all'inizio dell'esecuzione ($t = 0$). Essendo la politica *Working Set* con $\Delta = 10$, la Pagina 1024 verrà espulsa molto prima che M la richieda alla fine del programma ($i = 1023$). Pertanto, la Pagina 1024 genererà un fault due volte.

I 10 frame allocati sono sufficienti poiché l'accesso sequenziale mantiene il Working Set effettivo ≤ 4 pagine in ogni istante, prevenendo il thrashing.

Totale Page Fault = 1025 (per M) + 514 (per V) = **1539** Page Fault.

Domanda 2 (RAID e Affidabilità)

A1) È possibile che $MTTR > MTTF$? SÌ.

Matematicamente è possibile che il tempo medio di riparazione superi il tempo medio prima di un guasto. Ingegneristicamente, tuttavia, descrive un sistema disfunzionale in cui i dischi si rompono più in fretta di quanto possano essere riparati, rendendo inutile la ridondanza e garantendo una perdita di dati quasi certa.

A2) È possibile che $MTTDL > 10 \times MTTF$? SÌ.

È esattamente lo scopo per cui è stata concepita l'architettura RAID. Tramite meccanismi di ridondanza (mirroring o parità), l'affidabilità dell'array scala esponenzialmente, portando il tempo medio di perdita dei dati (MTTDL) a superare il ciclo di vita del singolo disco di svariati ordini di grandezza.

A3) Calcolo MTTR per RAID 1:

Formula teorica per RAID 1 a due dischi: $MTTDL = \frac{MTTF^2}{2 \cdot MTTR}$.

Imponiamo il vincolo richiesto:

$$\frac{MTTF^2}{2 \cdot MTTR} > 100 \cdot MTTF \implies \frac{MTTF}{2 \cdot MTTR} > 100 \implies MTTR < \frac{MTTF}{200}$$

Sostituendo $MTTF = 20.000$ ore:

$$MTTR < \frac{20.000}{200} \implies \mathbf{MTTR < 100 \text{ ore}}$$

Domanda 3 (I/O Sincrono, Asincrono e Polling)

A1) Sincrono vs Asincrono e Bloccante/Non-bloccante:

- **I/O Sincrono:** Il controllo torna al processo solo quando l'operazione hardware è fisicamente conclusa.
- **I/O Asincrono:** La chiamata ritorna immediatamente, innescando l'operazione in background. Il processo viene notificato al completamento effettivo.

Bloccante e Non-bloccante NON sono sinonimi di Sincrono e Asincrono. Sono sottocategorie dell'I/O sincrono. L'I/O non-bloccante ritorna subito se i dati non sono pronti (es. `EAGAIN`), ma **non** avvia un recupero in background.

A2) Polling vs Interrupt:

L'I/O sincrono può essere gestito sia in Polling (busy waiting della CPU sul registro di stato) sia in Interrupt (il thread dorme e viene risvegliato). L'I/O asincrono **deve** obbligatoriamente utilizzare gli Interrupt (spesso con DMA) poiché la CPU nel frattempo sta eseguendo altre istruzioni del processo e necessita di un segnale hardware asincrono per accorgersi del termine dell'operazione.

A3) Benefici e dipendenze dati:

Il vantaggio dell'I/O asincrono è l'**overlap** (sovrapposizione) tra il *CPU burst* (tempo di calcolo) e l'*I/O burst* (tempo di accesso al disco/rete), che massimizza il throughput del processo.

È possibile scrivere istruzioni dipendenti dai dati asincroni, ma non sequenzialmente. Esse devono essere confinate all'interno di un *Signal Handler* o di una funzione di *Callback* invocata asincronamente dal kernel a operazione conclusa.

Domanda 4 (System Call `sys_read` e Sicurezza)

B1) Chiamata diretta da user space: NO.

La funzione risiede nello spazio privilegiato del kernel (`kseg0`). I processi utente (`kuseg`) non hanno i permessi hardware per chiamarla direttamente. Devono utilizzare l'istruzione assembly `syscall` per generare una trap e demandare il cambio di contesto al gestore delle eccezioni.

B2) Accesso scorretto a memoria e correzione:

L'errore sta nell'effettuare un casting brutale (`char *`)`buf` del parametro fornito dall'utente e passarlo a una funzione interna (`kgets`). Se l'utente fornisce un puntatore al kernel space, il kernel sovrascriverebbe se stesso.

Correzione: Allocare un buffer temporaneo all'interno della `sys_read`, leggere i dati in quel buffer tramite `kgets`, e poi trasferirli in sicurezza verso il puntatore utente usando la funzione `copyout()`, che verifica rigorosamente la validità dell'indirizzo.

B3) Valore di ritorno rispetto a `size`:

Il ritorno diretto di `size` in questo codice è puramente indicativo di un'implementazione "giocattolo". In un'implementazione reale, il valore **può essere minore** (Short Read) a causa di End-of-File o input da console (line buffering). Il valore non può **mai essere maggiore** di `size`, altrimenti si genererebbe un *Buffer Overflow* nello spazio di memoria dell'utente.

Domanda 5 (Sincronizzazione Multi-core e Semantica dei Semafori)

B1) Disabilitazione interrupt su Multi-core:

Nei sistemi multi-processore, la disabilitazione degli interrupt altera unicamente il flag hardware del singolo core che esegue l'istruzione. Mentre la CPU locale blocca lo scheduler, le altre CPU

continuano a girare liberamente, permettendo ad altri thread di eseguire codice parallelo ed entrare nella stessa sezione critica.

B2) Analisi del codice dei semafori (P e V):

1. **Spinlock:** Garantisce l'atomicità hardware (mutua esclusione stretta) per l'accesso e l'aggiornamento simultaneo del contatore `sem_count` e della gestione della coda nel `sem_wchan`.
2. **Ciclo while (Semantica Mesa):** Il thread svegliato viene messo in stato *Ready*, non in *Run*. Prima che ottenga effettivamente la CPU, un terzo thread potrebbe rubargli la risorsa (`sem_count` torna a 0). Il ciclo forza il ri-controllo della condizione.
3. **Parametro spinlock in wchan_sleep:** Previene i *Lost Wakeup* e i *Deadlock*. Passando lo spinlock, il kernel si occupa di rilasciarlo in modo strettamente atomico nell'esatto istante in cui il thread viene inserito nella coda di sospensione.
4. **wchan_wakeone:** Sveglia esattamente **un solo** thread dalla coda di attesa.

B3) Condition Variable vs Wait Channel:

Il **Wait Channel** è un meccanismo primitivo del kernel OS161: è semplicemente la struttura dati (la coda) usata per sospendere e mantenere traccia dei thread dormienti. La **Condition Variable** è un costrutto di sincronizzazione di alto livello che incapsula al suo interno un `wait_channel`, ma aggiunge la logica applicativa per sospendere i processi in attesa che si verifichi una determinata espressione di stato protetta da un Lock esterno.